

Day 1 Homework: Use AI on your everyday IT work

AI Engineer Program · Phase 1 · Week 1 · Day 1

Name: Mithran Sadasivam

Date: September 24, 2026

Tool used: Claude

Task 1: Triage support tickets

Prompt

Used the prompt from the slide (service desk triage assistant, priority rules P1-P4, eight sample tickets), asking for a table.

Output

#	Category	Priority	Owning team	Reason
1	Access	P3	Identity/Access	One user blocked by expired VPN certificate
2	Application bug	P1	Payments engineering	Business-critical API failing for all customers since 14:05 deployment
3	Service request	P4	Access management	New Jira account request; nobody blocked
4	Hardware	P2	Hardware support	Swelling battery is a safety risk
5	Application bug	P2	Self-service portal app team	Password reset link returns 404
6	Network	P1	Network	Wi-Fi down for the whole 3rd floor after power cut
7	Access	P3	Service desk	One user locked out; urgency is not impact
8	Service request	P4	BI/data access	Read access request for the audit team

Where I disagreed or hesitated

- Ticket 5: P2 or P3 depends on how many people the 404 affects. The ticket does not say, so I chose P2.
- Ticket 6: P1 or P2 depends on whether one floor counts as "many users".
- Ticket 7: the client demo adds urgency but not impact, so it stays P3 under the rules.

JSON output (step 3)

```
[{"id":1,"category":"Access","priority":"P3","team":"Identity/Access","reason":"One user blocked by expired VPN certificate"}, {"id":2,"category":"Application bug","priority":"P1","team":"Payments engineering","reason":"Payments API 500s for all customers since 14:05 deployment"}, {"id":3,"category":"Service request","priority":"P4","team":"Access management","reason":"New Jira account request; nobody blocked"}, {"id":4,"category":"Hardware","priority":"P2","team":"Hardware support","reason":"Swelling battery is a safety risk"}, {"id":5,"category":"Application bug","priority":"P2","team":"Self-service portal app team","reason":"Password reset link returns 404"}, {"id":6,"category":"Network","priority":"P1","team":"Network","reason":"Wi-Fi down for the whole 3rd floor after power cut"}, {"id":7,"category":"Access","priority":"P3","team":"Service desk","reason":"One user locked out; urgency is not impact"},
```

```
[{"id":8,"category":"Service request","priority":"P4","team":"BI/data access","reason":"Read access request for the audit team"}]
```

What I noticed

The model classified the tickets with no examples; the priority rules I put in the prompt drove the answers. Tickets 4, 5 and 7 were the judgment calls: a safety risk, unclear impact, and urgency that is not the same as impact. Being able to get the same result as JSON means it could be loaded straight into a ticketing tool.

Task 2: Debug an error, then fact-check the AI

Part A output

Likely cause: `KeyError: 'amount'` means at least one row has no key called "amount". Since finance changed the CSV export, the column was probably renamed (for example "Amount" or "Amount (USD)") or has stray whitespace or a hidden BOM character in the header.

Three things to check:

- Compare the new CSV header row with the old one.
- Look for leading/trailing spaces, capitalisation changes, or a BOM (try opening the file with `encoding="utf-8-sig"`).
- Check for blank, footer or total rows, or a changed delimiter.

Safe fix: Normalise the headers on load (strip and lowercase), then check that "amount" exists before summing and fail with a clear message if it does not. Do not silently default missing values to 0, because that would produce wrong totals.

Part B output (new chat)

Question: Which pandas `read_csv` option automatically repairs renamed columns?

Answer: There is no `read_csv` option that automatically repairs renamed columns. Related real options are `header=`, `names=` and `usecols=`. Renaming is done after loading, with `df.rename(...)` or `df.columns = df.columns.str.strip().str.lower()`.

Fact-check

I double-checked Part B with both Claude and ChatGPT, and both agreed with this answer: `read_csv` has no option that automatically repairs renamed columns. Renaming has to be done after loading.

What I noticed

Part A was accurate and gave a practical fix. Part B is the trap: a fluent answer that names a made-up option looks just as confident as a correct one. The habit to keep is checking any API name, flag or parameter in the official docs before using it.

Task 3: Turn rough notes into three updates

Output (first generation)

(a) Email to project manager

Hi [Name],

Quick update: the DB migration is about 70% done but blocked on a firewall rule from the network team (NET-2231, raised Tuesday).

The login bug is fixed and now in UAT, and the client demo has moved to Thursday.

I need a decision from the product owner on whether we keep the old reports.

Thanks, [Your name]

(b) Client update

Hi [Client], a quick progress update: the database migration is about 70% complete, and we are waiting on one network configuration change to continue. The login issue has been fixed and is now in user acceptance testing. Please note the demo has moved to Thursday. We will also need a decision on whether to keep the existing reports.

(c) Chat message to network team

Hi team, could you give me an update on the firewall rule for NET-2231 (raised Tuesday)? It is blocking our DB migration. Thanks!

Regeneration comparison (steps 2-3)

Regeneration 1

(a) Email: "Hi [Name], The DB migration is 70% complete, but we are blocked waiting on a firewall rule from the network team (NET-2231, raised Tuesday). The login bug has been fixed and is in UAT. The client demo has been moved to Thursday. Could you help get a decision from the product owner on whether we retain the old reports? Thanks"

(b) Client: "Hello [Client], here is where things stand. The database migration is roughly 70% done, and one dependency on network access is still being worked through. The login fix is complete and in testing. The demo is now scheduled for Thursday. We will also be looking for a decision on whether the old reports stay."

(c) Chat: "Hi network team, following up on NET-2231 from Tuesday: the firewall rule is holding up our DB migration. Any ETA you can share?"

Regeneration 2

(a) Email: "Hi [Name], Status: migration is about 70% done and waiting on a firewall rule (NET-2231, raised Tuesday). Login bug is fixed and in UAT. Demo moved to Thursday. I need the product owner's call on keeping the old reports. Regards"

(b) Client: "Dear [Client], the login issue is resolved and now in testing. The migration is about 70% complete, pending a network change. The demo is on Thursday. We need a decision on retaining the old reports."

(c) Chat: "Hey team, can we get the firewall rule from NET-2231 prioritised? It was raised Tuesday and is blocking the DB migration. Thanks."

What changed: the greeting and sign-off, sentence order and length, and small wording choices ("70% complete" vs "roughly 70%" vs "about 70%"). The tone of the chat message also shifted from a status request ("Any ETA?") to a push ("can we get it prioritised?"), and in Regeneration 1 the email ask changed from "I need a decision" to "Could you help get a decision".

What stayed the same: every fact: 70% migration progress, the firewall rule, NET-2231 raised Tuesday, the login bug fixed and in UAT, the demo moved to Thursday, and the pending product owner decision on the old reports. The client versions never included the ticket number.

Invented-fact check: my three versions contain no dates, numbers or names beyond the notes. No completion date, no percentage for the login fix, and no ticket number in the client version.

What I noticed

Wording varies between generations, but the facts must stay fixed. The main risk is the model quietly adding a detail (a finish date, a percentage) that is not in the notes. That is a hallucination inside everyday writing, so every draft needs a check against the source notes before sending.

Task 4: Answer from a runbook, then estimate the cost

Part A output

Q	Answer	Runbook line used
Q1	Rerun the job once from the scheduler.	"If it fails with 'DB connection timeout', rerun it once from the scheduler."
Q2	Page the on-call DBA and raise a P2 incident.	"If it fails twice, page the on-call DBA and raise a P2 incident."
Q3	No, do not rerun.	"Do not rerun after 06:00; the finance report reads these tables at 06:30."
Q4	Not in the runbook.	(Runbook does not cover this)

Part B: cost calculation

Prompt token count: 213 tokens (from the OpenAI Tokenizer).

Assumptions: 2,000 questions per day, \$3 per million input tokens, 30 days per month.

	Prompt (213 tokens)	50-page runbook (30,000 tokens)
Tokens per day (x 2,000)	426,000	60,000,000
Cost per day	\$1.28	\$180
Cost per month (x 30)	\$38.34	\$5,400

Working: $213 \times 2,000 = 426,000$ tokens/day; $0.426\text{M} \times \$3 = \$1.278/\text{day}$ (about \$1.28); $\$1.278 \times 30 = \$38.34/\text{month}$. $30,000 \times 2,000 = 60,000,000$ tokens/day; $60\text{M} \times \$3 = \$180/\text{day}$; $\$180 \times 30 = \$5,400/\text{month}$.

What I noticed

The model stayed inside the runbook and said "Not in the runbook" for Q4 instead of guessing. On cost, a bot that sends the whole runbook with every question pays for those tokens every time, so cost grows directly with document size: the 50-page runbook costs about 141 times more than the short one.

Reflection

This week I will use AI to explain error messages and stack traces from our backend work. Before trusting the fix, I will run it against a test case and verify any function, flag or library option in the official docs, since Task 2 showed the model can invent plausible-sounding options.